**What is K-Means Clustering?**

K-means clustering is a popular unsupervised machine learning algorithm used for partitioning a dataset into a pre-defined number of clusters. The goal is to group similar data points together and discover underlying patterns or structures within the data.

Recall the first property of clusters — it states that the points within a cluster should be similar to each other. So, our aim here is to minimize the distance between the points within a cluster.

There is an algorithm that tries to minimize the distance of the points in a cluster with their centroid — the k-means clustering technique.

K-means is a centroid-based algorithm or a distance-based algorithm, where we calculate the distances to assign a point to a cluster. In K-Means, each cluster is associated with a centroid.

Optimization plays a crucial role in the k-means clustering algorithm. The goal of the optimization process is to find the best set of centroids that minimizes the sum of squared distances between each data point and its closest centroid.

**How K-Means Clustering Works?**

Here's how it works:

1. **Initialization**: Start by randomly selecting K points from the dataset. These points will act as the initial cluster centroids.

2. **Assignment**: For each data point in the dataset, calculate the distance between that point and each of the K centroids. Assign the data point to the cluster whose centroid is closest to it. This step effectively forms K clusters.

3. **Update centroids**: Once all data points have been assigned to clusters, recalculate the centroids of the clusters by taking the mean of all data points assigned to each cluster.

4. **Repeat**: Repeat steps 2 and 3 until convergence. Convergence occurs when the centroids no longer change significantly or when a specified number of iterations is reached.

5. **Final Result**: Once convergence is achieved, the algorithm outputs the final cluster centroids and the assignment of each data point to a cluster.

**Objective of k means Clustering**

The main objective of k-means clustering is to partition your data into a specific number (k) of groups, where data points within each group are similar and dissimilar to points in other groups. It achieves this by minimizing the distance between data points and their assigned cluster's center, called the centroid.

Here's an objective:

- **Grouping similar data points:** K-means aims to identify patterns in your data by grouping data points that share similar characteristics together. This allows you to discover underlying structures within the data.
- **Minimizing within-cluster distance:** The algorithm strives to make sure data points within a cluster are as close as possible to each other, as measured by a distance metric (usually Euclidean distance). This ensures tight-knit clusters with high cohesiveness.
- **Maximizing between-cluster distance:** Conversely, k-means also tries to maximize the separation between clusters. Ideally, data points from different clusters should be far apart, making the clusters distinct from each other.

**What is Clustering?**

Cluster analysis is a technique in **data mining** and **machine learning** that groups similar objects into clusters. K-means clustering, a popular method, aims to divide a set of objects into K clusters, minimizing the sum of squared distances between the objects and their respective cluster centers.

Hierarchical clustering and k-means clustering are two popular techniques in the field of unsupervised learning used for clustering data points into distinct groups. While k-means clustering divides data into a predefined number of clusters, hierarchical clustering creates a hierarchical tree-like structure to represent the relationships between the clusters.

**Example of Clustering**

Let’s try understanding this with a simple example. A bank wants to give credit card offers to its customers. Currently, they look at the details of each customer and, based on this information, decide which offer should be given to which customer.

Now, the bank can potentially have millions of customers. Does it make sense to look at the details of each customer separately and then make a decision? Certainly not! It is a manual process and will take a huge amount of time.

So what can the bank do? One option is to segment its customers into different groups. For instance, the bank can group the customers based on their income:

Press enter or click to view image in full size

Can you see where I'm going with this? The bank can now make three different strategies or offers, one for each group. Here, instead of creating different strategies for individual customers, they only have to make 3 strategies. This will reduce the effort as well as the time.

Think about it for a moment and use the example we just saw. Got it? Clustering is an unsupervised learning problem!

**How is Clustering an Unsupervised Learning Problem?**

Let's say you are working on a project where you need to predict the sales of a big mart:

| Outlet_Size | Outlet_Location_Type | Outlet_Type | Item_Outlet_Sales |
|---|---|---|---|
| Medium | Tier 1 | Supermarket Type1 | 3735.1380 |
| Medium | Tier 3 | Supermarket Type2 | 443.4228 |
| Medium | Tier 1 | Supermarket Type1 | 2097.2700 |
| NaN | Tier 3 | Grocery Store | 732.3800 |
| High | Tier 3 | Supermarket Type1 | 994.7052 |

Or, a project where your task is to predict whether a loan will be approved or not:

| Loan_ID | Gender | Married | ApplicantIncome | LoanAmount | Loan_Status |
|---|---|---|---|---|---|
| LP001002 | Male | No | 5849 | 130.0 | Y |
| LP001003 | Male | Yes | 4583 | 128.0 | N |
| LP001005 | Male | Yes | 3000 | 66.0 | Y |
| LP001006 | Male | Yes | 2583 | 120.0 | Y |
| LP001008 | Male | No | 6000 | 141.0 | Y |

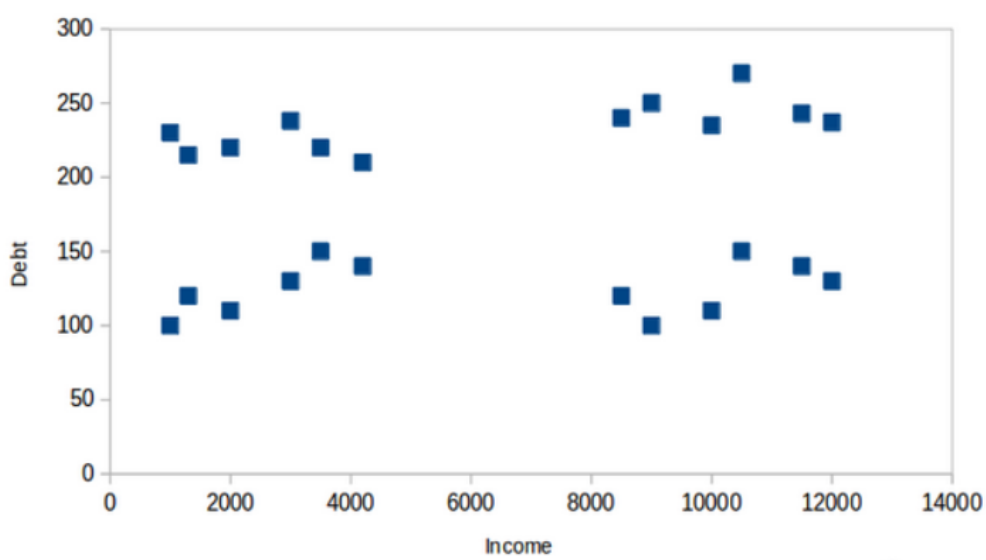
We have a fixed target to predict in both of these situations. In the sales prediction problem, we have to predict the *Item_Outlet_Sales* based on *outlet_size, outlet_location_type*, etc., and in the loan approval problem, we have to predict the *Loan_Status* depending on the Gender, marital status, the income of the customers, etc.

We now know what clusters are and the concept of clustering. Next, let's look at the properties of these clusters, which we must consider while forming the clusters.

**Properties of K means Clustering**

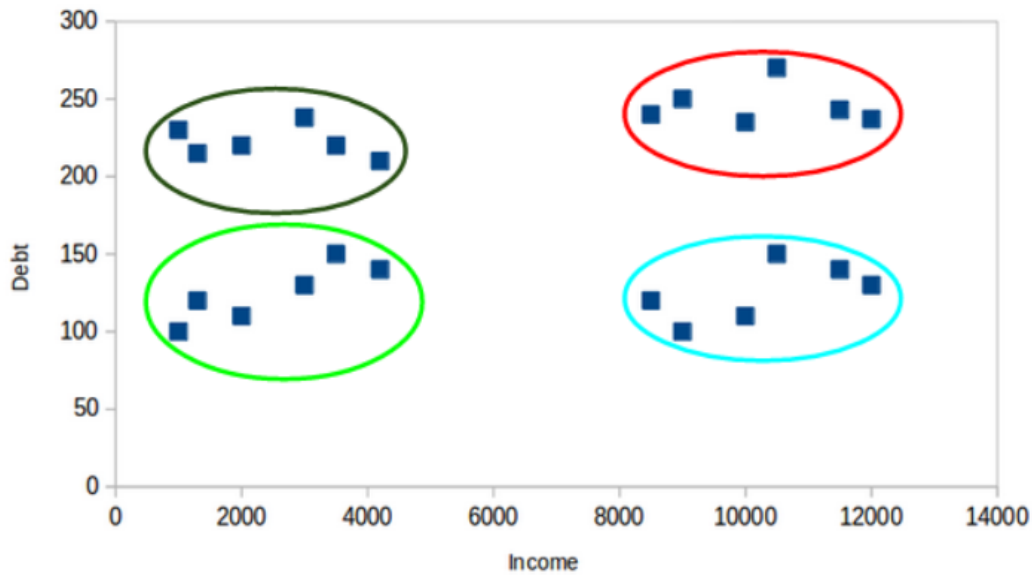
How about another example of k-means clustering algorithm? We'll take the same bank as before, which wants to segment its customers. For simplicity purposes, let's say the bank only wants to use the income and debt to make the **segmentation**. They collected the customer data and used a scatter plot to visualize it:

Press enter or click to view image in full size



On the X-axis, we have the income of the customer, and the y-axis represents the amount of debt. Here, we can clearly visualize that these customers can be segmented into 4 different clusters, as shown below:

Press enter or click to view image in full size




This is how clustering helps to create segments (clusters) from the data. The bank can further use these clusters to make strategies and offer discounts to its customers. So let's look at the properties of these clusters.

**First Property of K-Means Clustering Algorithm**

All the data points in a cluster should be similar to each other. Let me illustrate it using the above example:

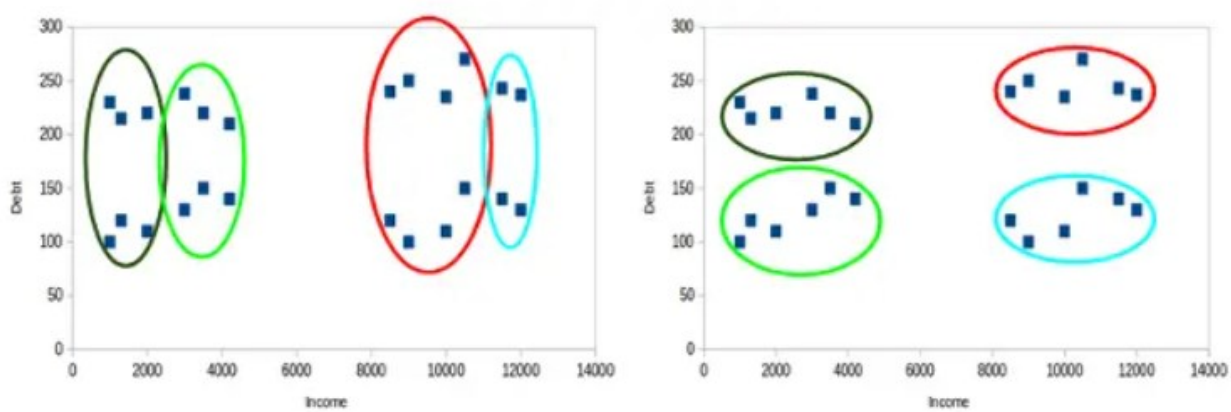
If the customers in a particular cluster are not similar to each other, then their requirements might vary, right? If the bank gives them the same offer, they might not like it, and their interest in the bank might reduce. Not ideal.

Having similar data points within the same cluster helps the bank to use targeted marketing. You can think of similar examples from your everyday life and consider how clustering will (or already does) impact the business strategy.

**Second Property of K-Means Clustering Algorithm**

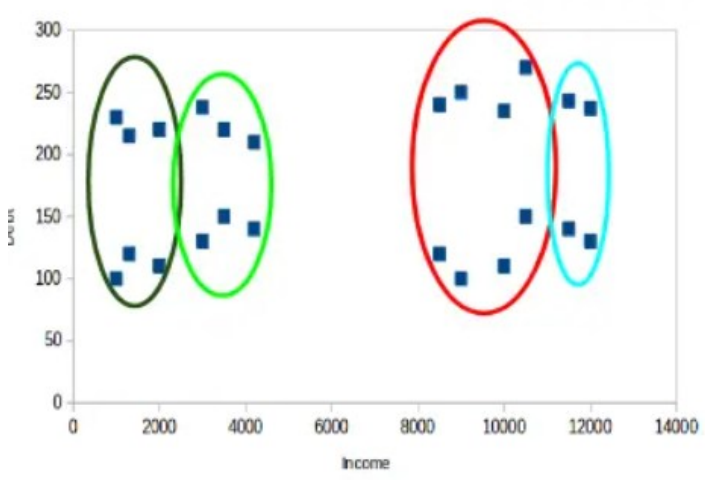
The data points from different clusters should be as different as possible. This will intuitively make sense if you've grasped the above property. Let's again take the same example to understand this property:

Press enter or click to view image in full size



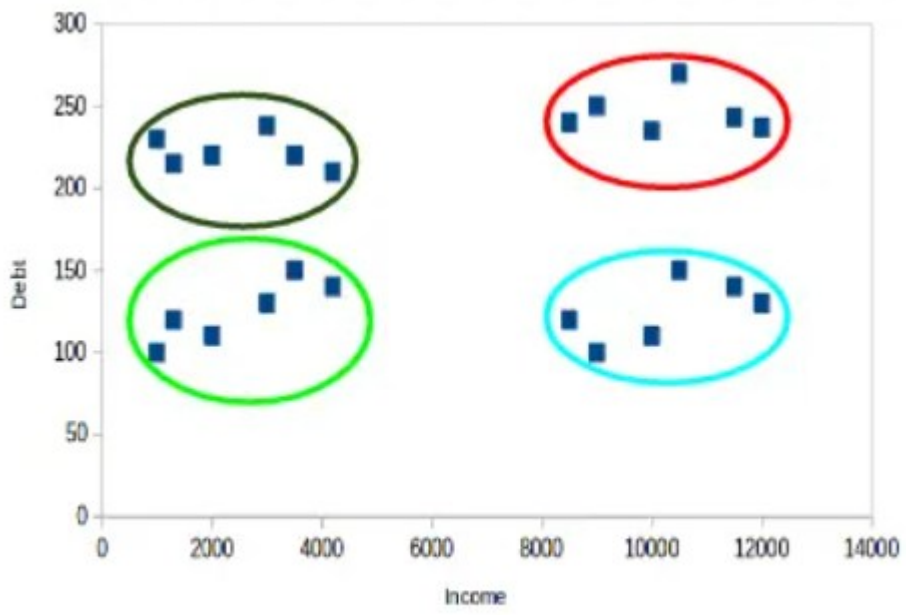

Case - I  Case - II

Which of these cases do you think will give us the better clusters? If you look at case I:




Case - I

Customers in the red and blue clusters are quite similar to each other. The top four points in the red cluster share similar properties to those of the blue cluster's top two customers. They have high incomes and high debt values. Here, we have clustered them differently. Whereas, if you look at case II:




Case - II

Points in the red cluster completely differ from the customers in the blue cluster. All the customers in the red cluster have high income and high debt, while the customers in the blue cluster have high income and low debt value. Clearly, we have a better clustering of customers in this case.

Hence, data points from different clusters should be as different from each other as possible to have more meaningful clusters. The k-means algorithm uses an iterative approach to find the optimal cluster assignments by minimizing the sum of squared distances between data points and their assigned cluster centroid.

**Applications of Clustering in Real-World Scenarios**

Clustering is a widely used technique in the industry. It is being used in almost every domain, from banking and **recommendation engines** to document clustering and image segmentation.

**Customer Segmentation**

We covered this earlier — one of the most common applications of clustering is **customer segmentation**. And it isn't just limited to banking. This strategy is across functions, including telecom, e-commerce, sports, advertising, sales, etc.

**Document Clustering**

This is another common application of clustering. Let's say you have multiple documents and you need to cluster similar documents together. Clustering helps us group these documents such that similar documents are in the same clusters.

**Image Segmentation**

We can also use clustering to perform image segmentation. Here, we try to club similar pixels in the image together. We can apply clustering to create clusters having similar pixels in the same group.

**Recommendation Engines**

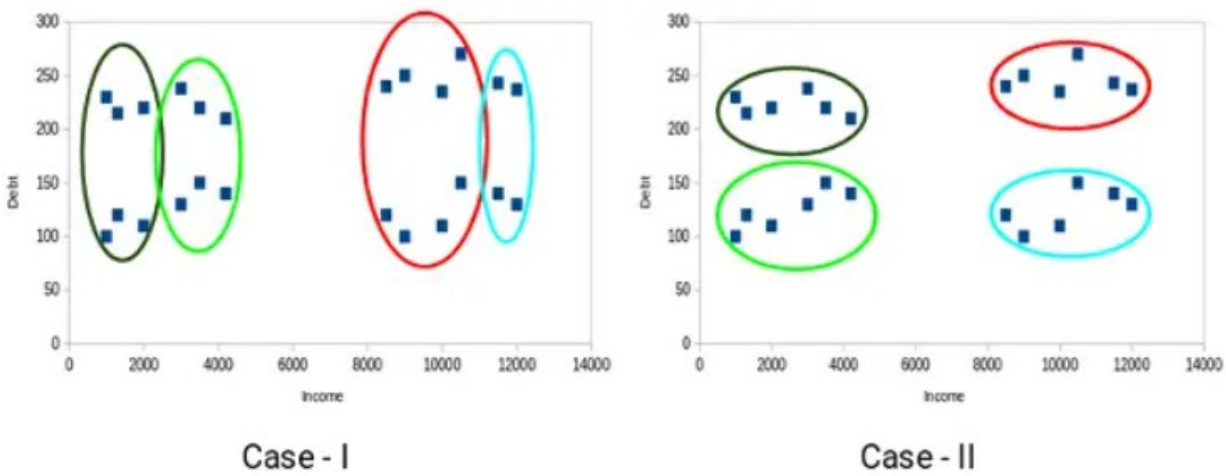
Clustering can also be used in recommendation engines. Let's say you want to recommend songs to your friends. You can look at the songs liked by that person and then use clustering to find similar songs and finally recommend the most similar songs.

There are many more applications that I'm sure you have already thought of. You can share these applications in the comments section below. Next, let's look at how we can evaluate our clusters.

**Understanding the Different Evaluation Metrics for Clustering**

The primary aim of clustering is not just to make clusters but to make good and meaningful ones. We saw this in the below example:

Press enter or click to view image in full size




Here, we used only two features, and hence it was easy for us to visualize and decide which of these clusters was better.

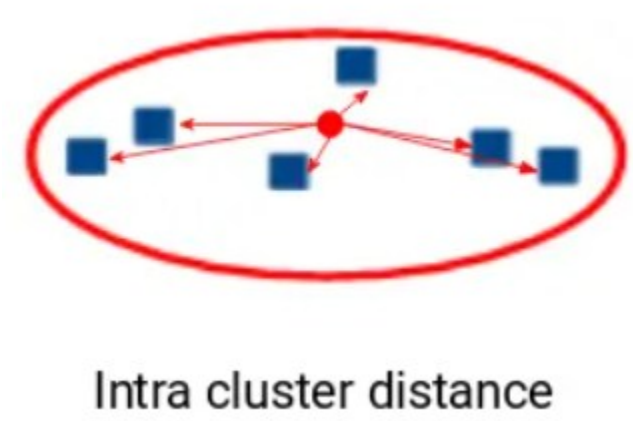
Unfortunately, that's not how real-world scenarios work. We will have a ton of features to work with. Let's take the customer segmentation example again — we will have features like customers' income, occupation, gender, age, and many more. We would not be able to visualize all these features together and decide on better and more meaningful clusters.

This is where we can make use of **evaluation metrics**. Let's discuss a few of them and understand how we can use them to evaluate the quality of our clusters.

**Inertia**

Recall the first property of clusters we covered above. This is what inertia evaluates. It tells us how far the points within a cluster are. So, inertia actually calculates the sum of distances of all the points within a cluster from the centroid of that cluster. Normally, we use Euclidean distance as the distance metric, as long as most of the features are numeric; otherwise, Manhattan distance in case most of the features are categorical.

We calculate this for all the clusters; the final inertial value is the sum of all these distances. This distance within the clusters is known as intracluster distance. So, inertia gives us the sum of intracluster distances:




Now, what do you think should be the value of inertia for a good cluster? Is a small inertial value good, or do we need a larger value? We want the points within the same cluster to be similar to each other, right? Hence, the distance between them should be as low as possible.

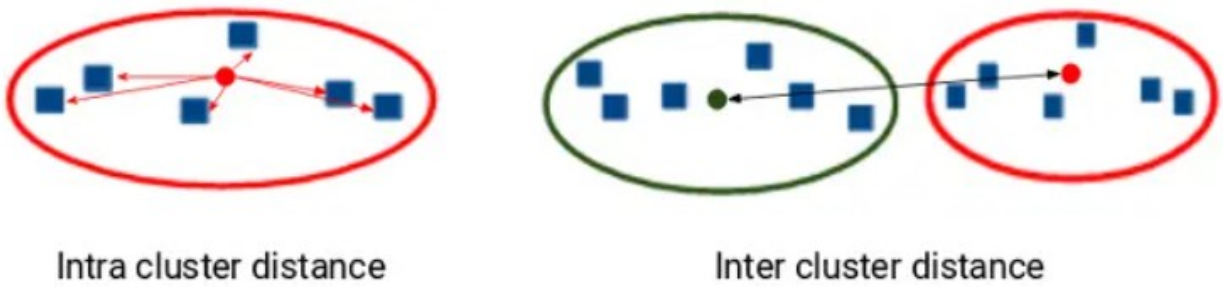
**Dunn Index**

We now know that inertia tries to minimize the intracluster distance. It is trying to make more compact clusters.

Let me put it this way — if the distance between the centroid of a cluster and the points in that cluster is small, it means that the points are closer to each other. So, inertia makes sure that the first property of clusters is satisfied. But it does not care about the second property — that different clusters should be as different from each other as possible.

This is where the Dunn index comes into action.

Press enter or click to view image in full size




Along with the distance between the centroid and points, the Dunn index also takes into account the distance between two clusters. This distance between the centroids of two different clusters is known as inter-cluster distance. Let's look at the formula of the Dunn index:

$$\text{Dunn Index} = \frac{\text{min(Inter cluster distance)}}{\text{max(Intra cluster distance)}}$$

We want to maximize the Dunn index. The more the value of the Dunn index, the better the clusters will be. Let's understand the intuition behind the Dunn index:

Clusters are far apart

$$\text{Dunn Index} = \frac{\text{min(Inter cluster distance)}}{\text{max(Intra cluster distance)}}$$

In order to maximize the value of the Dunn index, the numerator should be maximum. Here, we are taking the minimum of the inter-cluster distances. So, the distance between even the closest clusters should be more which will eventually make sure that the clusters are far away from each other.

$$\text{Dunn Index} = \frac{\text{min(Inter cluster distance)}}{\text{max(Intra cluster distance)}}$$

Clusters are compact

Also, the denominator should be minimum to maximize the Dunn index. Here, we are taking the maximum of all intracluster distances. Again, the intuition is the same here. The maximum distance between the cluster centroids and the points should be minimum, eventually ensuring that the clusters are compact.

**Silhouette Score**

The silhouette score and plot are used to evaluate the quality of a clustering solution produced by the k-means algorithm. The silhouette score measures the similarity of each point to its own cluster compared to other clusters, and the silhouette plot visualizes these scores for each sample. A high silhouette score indicates that the clusters are well separated, and each sample is more similar to the samples in its own cluster than to samples in other clusters. A silhouette score close to 0 suggests overlapping clusters, and a negative score suggests poor clustering solutions.

**How to Apply K-Means Clustering Algorithm?**

Let's now take an example to understand how K-Means actually works:

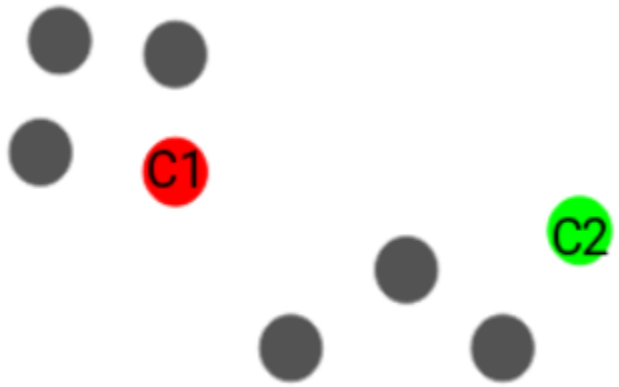
We have these 8 points, and we want to apply k-means to create clusters for these points. Here's how we can do it.

1. **Choose the number of clusters *k***

The first step in k-means is to pick the number of clusters, k.

**2. Select k random points from the data as centroids**

Next, we randomly select the centroid for each cluster. Let's say we want to have 2 clusters, so k is equal to 2 here. We then randomly select the centroid:

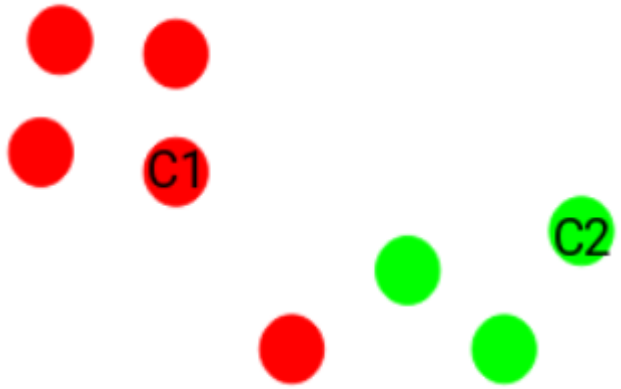



Here, the red and green circles represent the centroid for these clusters.

**3. Assign all the points to the closest cluster centroid**

Once we have initialized the centroids, we assign each point to the closest cluster centroid:

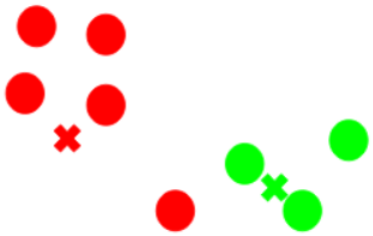
Press enter or click to view image in full size




Here you can see that the points closer to the red point are assigned to the red cluster, whereas the points closer to the green point are assigned to the green cluster.

**4. Recompute the centroids of newly formed clusters**

Now, once we have assigned all of the points to either cluster, the next step is to compute the centroids of newly formed clusters:



Here, the red and green crosses are the new centroids.

**5. Repeat steps 3 and 4**

We then repeat steps 3 and 4:

*The step of computing the centroid and assigning all the points to the cluster based on their distance from the centroid is a single iteration*. But wait — when should we stop this process? It can't run till eternity, right?

**Stopping Criteria for K-Means Clustering**

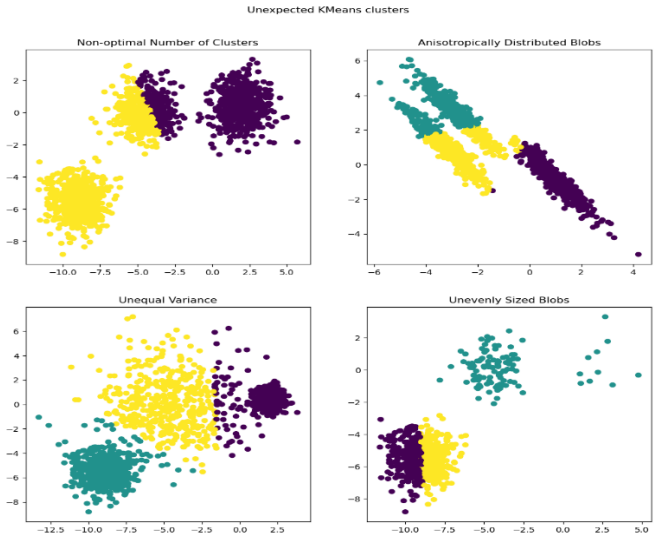
There are essentially three stopping criteria that can be adopted to stop the K-means algorithm:

1. Centroids of newly formed clusters do not change
2. Points remain in the same cluster
3. Maximum number of iterations is reached

We can stop the algorithm if the centroids of newly formed clusters are not changing. Even after multiple iterations, if we are getting the same centroids for all the clusters, we can say that the algorithm is not learning any new pattern, and it is a sign to stop the training.

Another clear sign that we should stop the training process is if the points remain in the same cluster even after training the algorithm for multiple iterations.

Finally, we can stop the training if the maximum number of iterations is reached. Suppose we have set the number of iterations as 100. The process will repeat for 100 iterations before stopping.




**Choosing the Number of Clusters (K) in K-Means**

Introduction

When using the K-Means algorithm, one requirement stands out: you need to specify the exact number of clusters, K, before running it.
This differs from supervised learning, where the number of categories is usually determined by the labels in the dataset.
In unsupervised learning, since there are no labels, determining the optimal number of groups (K) is a key challenge.

Why Choosing the Right K is Important

- Too small K → distinct groups may be merged → loss of meaningful structure.
- Too large K → natural groups may be split into smaller, less meaningful fragments.
- Choosing a suitable K ensures the clusters discovered reflect the underlying structure of the data effectively.

Common Methods to Choose K

1. Using Domain Knowledge

- If you already have some understanding of the data or the problem, you can guide the selection of K.
- Example: Clustering retail customers into:
    - High spenders

    - Occasional shoppers
    - Inactive users
      → Here, prior knowledge suggests K = 3.

- Not always possible with new datasets, but existing knowledge can help.

Python example:

```
from sklearn.cluster import KMeans

K = 3  # determined based on business knowledge
kmeans = KMeans(n_clusters=K, random_state=42)
kmeans.fit(data)
labels = kmeans.labels_
```

---

2. Visual Inspection (for low-dimensional data)

- Works best for 2–3 features (dimensions).
- A scatter plot can reveal natural clusters visually.
- Limitation: Most datasets have more dimensions, and visual interpretation is subjective.

Python example:

```
import matplotlib.pyplot as plt

plt.scatter(data[:, 0], data[:, 1], c='blue', s=50)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Scatter Plot for Visual Inspection')
plt.show()
```

---

3. The Elbow Method

- Run K-Means for a range of K values (e.g., 1–10).
- Compute Within-Cluster Sum of Squares (WCSS) for each K:
    - WCSS = sum of squared distances between each point and its cluster centroid.
    - Smaller WCSS → points are closer to their cluster centers → tighter clusters.
- Plot WCSS vs. K → look for the "elbow": the point where WCSS decrease starts slowing down.
    - This represents a balance: adding more clusters beyond this point gives diminishing returns.

- Important Note: The elbow is not always sharp. Sometimes the curve is smooth, making the choice ambiguous. In such cases, combine this method with domain knowledge or other techniques.

Python example:

```python
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(data)
    wcss.append(kmeans.inertia_)  # inertia_ is WCSS

plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel('Number of clusters (K)')
plt.ylabel('WCSS')
plt.title('Elbow Method')
plt.show()
```

4. Silhouette Score

- Measures how well each point fits its cluster compared to other clusters.

- Silhouette coefficient for a point:
  [
  s = \frac{b - a}{\max(a, b)}
  ]
  - a = mean distance to other points in the same cluster (intra-cluster distance)
  - b = mean distance to points in the nearest other cluster
- Interpretation:
  - s ≈ +1 → point is well inside its cluster
  - s ≈ 0 → point is near a cluster boundary
  - s ≈ -1 → point may be assigned to the wrong cluster
- Average silhouette score over all points can guide K selection.

Python example:

```python
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

sil_scores = []
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(data)
    score = silhouette_score(data, labels)
    sil_scores.append(score)

plt.plot(range(2, 11), sil_scores, marker='o')
plt.xlabel('Number of clusters (K)')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Method')
plt.show()
```

---

Summary

- As K increases, WCSS decreases, but if K equals the number of data points, WCSS = 0 → not meaningful.
- Optimal K often requires combining:
    - Quantitative methods (Elbow, Silhouette)
    - Domain knowledge
    - Analysis goals

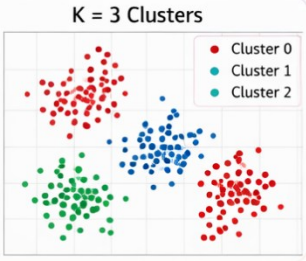
# Choosing the Number of Clusters (K) in K-Means

## Code Execution Results

### 1. Using Domain Knowledge

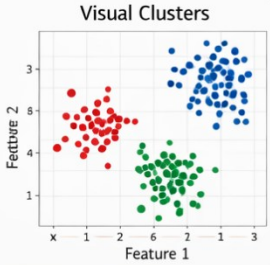
```
K = 3
kmeans = KMeans(n_clusters=3)
kmeans.fit(data)
labels = kmeans.labels_
```

**Result:**
Data clustered into 3 groups.



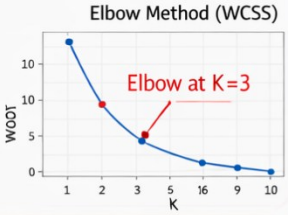

### 2. Visual Inspection (Scatter Plot)

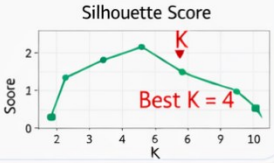
```
plt.scatter(data[:,0], data[:,1])
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Visual Clusters')
```




### 3. Elbow Method

```
wcss = []
for k in range(1,11):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(data)
    wcss.append(kmeans.inertia_)
```




### 4. Silhouette Score

```
sil_scores = []
for k in range(2,11):
    labels = KMeans(k).fit_predict(data)
    score = silhouette_score(data, labels)
    sil_scores.append(score)
```